UNIVASSOURAS
ENGENHARIA DE SOFTWARE

LEANDRO LIMA CARDOSO (202323366)

DESENVOLVIMENTO DE UMA *GAME ENGINE* 2D EM FORMATO DE BIBLIOTECA EM C++

PRÁTICAS EXTENSIONISTAS INTEGRADORAS VI

SAQUAREMA
2026

LEANDRO LIMA CARDOSO (202323366)

DESENVOLVIMENTO DE UMA *GAME ENGINE* 2D EM FORMATO DE BIBLIOTECA EM C++

PRÁTICAS EXTENSIONISTAS INTEGRADORAS VI

Trabalho apresentado como requisito parcial para obtenção de nota da disciplina de **Práticas Extensionistas Integradoras VI** do curso de Engenharia de Software da Univassouras (campus universitário de Saquarema).

Prof. Dr. Altemar Sales

SAQUAREMA

2026

RESUMO

Este trabalho apresenta o desenvolvimento de uma *game engine* 2D implementada em C++, destinada à criação de jogos baseados em *tiles* com visão *top-down*. O projeto tem como objetivo estruturar uma arquitetura modular e performática, minimizando a dependência de bibliotecas externas e proporcionando maior controle sobre gerenciamento de memória, renderização gráfica e organização de recursos. A pesquisa caracteriza-se como aplicada e exploratória, fundamentada em revisão bibliográfica sobre engenharia de software, padrões de projeto e arquitetura de sistemas. A metodologia adotada envolve modelagem arquitetural, implementação incremental dos módulos da engine e validação por meio da construção de um protótipo funcional de jogo. São abordados aspectos como sistema de renderização 2D, gerenciamento de entidades, controle de colisões e administração de *assets*. Como resultado esperado, busca-se obter uma *engine* leve, especializada e de fácil utilização para jogos estruturados em mapas compostos por *tiles*, contribuindo para o aprofundamento acadêmico em arquitetura de software e otimização de desempenho em aplicações gráficas.

Palavras-chave: *game engine*; C++; arquitetura de software; jogos 2D; desempenho.

SUMÁRIO

1 INTRODUÇÃO 5

1.1 Contexto 6

1.2 Problema 6

1.3 Justificativa 7

1.4 Objetivos 7

1.4.1 Objetivo Geral 7

1.4.2 Objetivos Específicos 8

2 REFERENCIAL TEÓRICO 8

2.1 Engenharia de Software e Arquitetura de Sistemas 8

2.2 Padrões de Projeto e Modularidade 9

2.3 *Game Engines*: Conceito e Estrutura 9

2.4 Renderização 2D e Sistemas Baseados em *Tiles* 10

2.5 Linguagem C++ e Desempenho 10

2.6 Gerenciamento de Recursos e Entidades 11

3 METODOLOGIA 11

3.1 Procedimentos Técnicos 12

3.2 Modelagem Arquitetural 12

3.3 Implementação Incremental 13

3.4 Minimização de Dependências Externas 14

3.5 Validação do Sistema 14

3.6 Limitações Metodológicas 15

REFERÊNCIAS 15

1 INTRODUÇÃO

A indústria de jogos digitais consolidou-se como um dos segmentos mais relevantes da indústria de software contemporânea, impulsionando avanços significativos nas áreas de computação gráfica, arquitetura de sistemas, processamento paralelo e otimização de desempenho. O desenvolvimento de jogos digitais demanda integração entre múltiplas disciplinas, incluindo engenharia de software, modelagem matemática, design de interação e programação orientada a objetos. Nesse contexto, as *game engines* assumem papel central, pois fornecem infraestrutura reutilizável responsável pela abstração de funcionalidades complexas como renderização gráfica, gerenciamento de memória, controle de entrada e organização de entidades.

Segundo Sommerville (2019), a definição adequada da arquitetura de software é um dos fatores mais relevantes para garantir qualidade, manutenibilidade e desempenho em sistemas complexos. No desenvolvimento de motores gráficos, decisões arquiteturais impactam diretamente na escalabilidade e eficiência do sistema. De forma complementar, Gamma et al. (1995) destacam que a aplicação adequada de padrões de projeto contribui para modularidade, reuso e organização estrutural do código, aspectos fundamentais em projetos que envolvem múltiplos subsistemas interdependentes.

Atualmente, motores amplamente utilizados no mercado, como Unity e Unreal Engine, oferecem soluções completas para desenvolvimento de jogos, incluindo sistemas gráficos avançados, física, animação e ferramentas visuais integradas. Entretanto, tais plataformas apresentam elevado nível de abstração, grande dependência de bibliotecas externas e arquitetura voltada para múltiplos tipos de jogos, o que pode resultar em complexidade excessiva para projetos específicos de menor escopo, especialmente no contexto educacional.

Nesse cenário, observa-se a necessidade de compreender de forma aprofundada os mecanismos internos que compõem uma *game engine*, particularmente no que se refere

ao gerenciamento de recursos, renderização 2D e estruturação modular. A linguagem C++ apresenta-se como alternativa adequada para esse propósito, devido ao seu controle refinado sobre memória e recursos computacionais, além de seu amplo uso em aplicações de alto desempenho (STROUSTRUP, 2013).

1.1 Contexto

Jogos 2D baseados em mapas compostos por *tiles* com visão *top-down* permanecem amplamente utilizados tanto em produções independentes quanto em ambientes acadêmicos. Esse modelo estrutural organiza o cenário em unidades gráficas discretas, permitindo controle eficiente de colisão, movimentação e renderização. A simplicidade conceitual desse formato torna-o apropriado para experimentação arquitetural e implementação incremental de funcionalidades típicas de motores gráficos.

Além disso, o desenvolvimento de uma *game engine* especializada em jogos 2D possibilita maior foco na otimização de subsistemas essenciais, como gerenciamento de entidades, controle de eventos, carregamento de recursos e organização de camadas de renderização, evitando a complexidade inerente a motores generalistas.

1.2 Problema

Apesar da disponibilidade de motores consolidados no mercado, observa-se que soluções generalistas frequentemente incorporam múltiplas camadas de abstração e dependências externas que não são estritamente necessárias para jogos 2D baseados em *tiles*. Essa característica pode dificultar o entendimento aprofundado da arquitetura interna do motor e limitar a experimentação acadêmica relacionada a desempenho e organização estrutural.

Dessa forma, formula-se o seguinte problema de pesquisa:

Como desenvolver uma *game engine* 2D em formato de biblioteca, utilizando C++, que seja modular, performática e com mínima dependência de bibliotecas externas, facilitando a criação de jogos baseados em *tiles* com visão *top-down*?

1.3 Justificativa

Do ponto de vista acadêmico, o desenvolvimento de uma *game engine* própria possibilita aplicação prática de conceitos fundamentais da engenharia de software, tais como definição arquitetural, modularização, encapsulamento, reuso de código e documentação técnica. Conforme destaca Sommerville (2019), sistemas bem estruturados arquiteturalmente tendem a apresentar maior facilidade de manutenção e evolução.

Sob a perspectiva técnica, a implementação direta de componentes essenciais como renderização 2D, gerenciamento de memória e organização de entidades, permite compreender detalhadamente o impacto das decisões arquiteturais no desempenho do sistema. Essa abordagem favorece o desenvolvimento de competências relacionadas à otimização de recursos computacionais, aspecto particularmente relevante em aplicações gráficas.

Adicionalmente, ao estruturar a *game engine* em formato de biblioteca, amplia-se sua possibilidade de reutilização em diferentes projetos, promovendo separação clara entre motor e aplicação final, característica alinhada às boas práticas de projeto de software (GAMMA et al., 1995).

1.4 Objetivos

1.4.1 Objetivo Geral

Desenvolver uma *game engine* 2D em formato de biblioteca, utilizando a linguagem C++, com foco em desempenho, modularidade e baixa dependência de bibliotecas externas, destinada à criação de jogos baseados em *tiles* com visão *top-down*.

1.4.2 Objetivos Específicos

- Projetar a arquitetura modular da *engine*, definindo camadas e responsabilidades.
- Implementar sistema de renderização 2D otimizado para mapas baseados em *tiles*.
- Desenvolver mecanismo de gerenciamento de recursos gráficos.
- Estruturar sistema de entidades e controle de colisão.
- Minimizar o uso de dependências externas, priorizando implementações próprias.
- Validar a *engine* por meio do desenvolvimento de um protótipo funcional.
- Documentar decisões arquiteturais e resultados obtidos.

2 REFERENCIAL TEÓRICO

2.1 Engenharia de Software e Arquitetura de Sistemas

A engenharia de software estabelece princípios e práticas voltados à construção de sistemas confiáveis, eficientes e manuteníveis. De acordo com Sommerville (2019), a arquitetura de software representa a estrutura fundamental de um sistema, definindo seus componentes, suas interações e as decisões de alto nível que impactam atributos de qualidade como desempenho, escalabilidade e manutenibilidade.

Em sistemas que envolvem processamento gráfico e manipulação intensiva de recursos, as decisões arquiteturais assumem papel ainda mais crítico. Pressman e Maxim (2020) destacam que a organização modular do software contribui para redução da

complexidade e facilita a evolução incremental do sistema, especialmente em projetos com múltiplos subsistemas interdependentes.

No contexto de motores gráficos, a arquitetura deve considerar separação clara entre camadas responsáveis por renderização, lógica de jogo, gerenciamento de recursos e tratamento de entrada, de modo a garantir baixo acoplamento e alta coesão entre componentes.

2.2 Padrões de Projeto e Modularidade

Os padrões de projeto constituem soluções consolidadas para problemas recorrentes no desenvolvimento de software. Segundo Gamma et al. (1995), os *design patterns* promovem reuso, organização estrutural e melhor comunicação entre desenvolvedores.

Em motores gráficos, padrões como *Singleton*, *Factory*, *Observer* e *Component* são frequentemente utilizados para estruturar subsistemas de gerenciamento global, criação de objetos, comunicação entre entidades e organização baseada em componentes. A aplicação adequada desses padrões favorece extensibilidade e manutenção, reduzindo dependência excessiva entre módulos.

A modularidade, conforme apontado por Sommerville (2019), permite dividir sistemas complexos em partes menores e independentes, tornando o desenvolvimento mais controlável e favorecendo testes isolados de funcionalidades.

2.3 *Game Engines*: Conceito e Estrutura

Uma *game engine* pode ser definida como um conjunto de bibliotecas e ferramentas responsáveis por fornecer infraestrutura reutilizável para o desenvolvimento de jogos digitais. Essas estruturas geralmente incluem sistemas de renderização, controle de entrada, física, áudio, gerenciamento de recursos e organização de entidades.

Motores amplamente difundidos como Unity e Unreal Engine apresentam arquiteturas generalistas, voltadas a múltiplos gêneros e plataformas. Embora ofereçam alto nível de abstração e produtividade, tais soluções podem incorporar complexidade significativa, tornando mais difícil a compreensão aprofundada de seus mecanismos internos.

Conforme Gregory (2018), motores gráficos modernos são estruturados em camadas que isolam subsistemas críticos, permitindo que o desenvolvedor concentre-se na lógica do jogo. Entretanto, a especialização para um domínio específico como jogos 2D baseados em *tiles*, pode possibilitar arquiteturas mais enxutas e direcionadas ao desempenho.

2.4 Renderização 2D e Sistemas Baseados em *Tiles*

A renderização 2D consiste na projeção de elementos gráficos bidimensionais em um plano cartesiano, organizados em camadas ou mapas. Em jogos baseados em *tiles*, o cenário é estruturado a partir de pequenas unidades gráficas repetíveis que compõem mapas maiores.

Esse modelo apresenta vantagens relacionadas à organização espacial, detecção de colisão e otimização de memória, uma vez que permite reutilização de recursos gráficos e simplificação do cálculo de posicionamento. Técnicas como *sprite batching* e controle de camadas são frequentemente utilizadas para minimizar chamadas de renderização e melhorar desempenho.

A organização em mapas discretos também facilita a implementação de sistemas de colisão baseados em grade, reduzindo complexidade computacional quando comparado a métodos contínuos.

2.5 Linguagem C++ e Desempenho

A linguagem C++ é amplamente empregada em sistemas que exigem alto desempenho e controle refinado sobre recursos computacionais. Conforme Stroustrup (2013), C++ combina abstração orientada a objetos com gerenciamento direto de memória, permitindo ao desenvolvedor equilibrar produtividade e eficiência.

No desenvolvimento de motores gráficos, o controle explícito de memória e a possibilidade de otimizações de baixo nível tornam C++ particularmente adequada. Além disso, sua compatibilidade com bibliotecas gráficas e APIs de baixo nível amplia a flexibilidade arquitetural.

O uso de C++ também favorece a implementação de bibliotecas reutilizáveis, permitindo encapsulamento de funcionalidades em módulos independentes e distribuíveis.

2.6 Gerenciamento de Recursos e Entidades

Motores gráficos necessitam administrar múltiplos recursos simultaneamente, como texturas, mapas, objetos e eventos. A organização adequada desses recursos influencia diretamente no desempenho e na estabilidade do sistema.

Abordagens modernas frequentemente adotam modelos baseados em componentes (*Entity Component System* – ECS), nos quais entidades são compostas por conjuntos de componentes independentes responsáveis por comportamentos específicos. Essa abordagem favorece flexibilidade e reduz acoplamento entre lógica e representação.

Além disso, técnicas de carregamento sob demanda e cache de recursos são estratégias comuns para evitar consumo excessivo de memória e melhorar tempo de resposta.

3 METODOLOGIA

Este trabalho caracteriza-se como uma pesquisa aplicada, uma vez que objetiva o desenvolvimento de um artefato tecnológico com finalidade prática, conforme classificação proposta por Gil (2019). Quanto aos objetivos, enquadra-se como pesquisa exploratória, pois busca aprofundar o conhecimento acerca da estrutura interna de uma *game engine* 2D e investigar soluções arquiteturais voltadas à otimização de desempenho.

No que se refere à abordagem, a pesquisa apresenta natureza predominantemente qualitativa, uma vez que se concentra na análise estrutural, organizacional e arquitetural do sistema desenvolvido, sem a aplicação de métodos estatísticos ou modelagens matemáticas formais.

3.1 Procedimentos Técnicos

Os procedimentos técnicos adotados envolveram:

- Pesquisa bibliográfica, com base em obras relacionadas à engenharia de software, arquitetura de sistemas, padrões de projeto e desenvolvimento de motores gráficos;
- Estudo conceitual sobre organização modular e gerenciamento de recursos em aplicações gráficas;
- Modelagem arquitetural do sistema antes da implementação;
- Desenvolvimento incremental da biblioteca proposta;
- Construção de protótipo funcional para validação prática.

A pesquisa bibliográfica fundamentou as decisões arquiteturais adotadas, especialmente no que se refere à modularização, baixo acoplamento e aplicação de padrões estruturais.

3.2 Modelagem Arquitetural

Antes da implementação, foi realizada a definição da arquitetura geral da *engine*, estabelecendo a separação entre os seguintes módulos principais:

- Núcleo da aplicação (*core*);
- Sistema de renderização 2D;
- Gerenciamento de recursos;
- Sistema de entidades;
- Controle de colisões;
- Interface de integração com o jogo.

Essa modelagem buscou garantir alta coesão interna e baixo acoplamento entre componentes, conforme princípios clássicos da engenharia de software (SOMMERVILLE, 2019). A definição prévia das responsabilidades de cada módulo permitiu reduzir retrabalho durante a fase de implementação.

3.3 Implementação Incremental

A implementação foi conduzida de forma incremental, priorizando inicialmente a estrutura básica da biblioteca e posteriormente adicionando funcionalidades específicas. A estratégia incremental foi adotada para permitir validação progressiva dos módulos implementados.

O desenvolvimento seguiu as seguintes etapas:

- Estruturação do núcleo da biblioteca e configuração do ambiente de compilação em C++;
- Implementação do sistema de renderização 2D voltado para mapas baseados em tiles;
- Desenvolvimento do sistema de gerenciamento de recursos gráficos;
- Implementação do sistema de entidades e lógica básica de movimentação;

- Integração entre os módulos e testes funcionais.

Essa abordagem permitiu isolamento de falhas e ajustes arquiteturais durante o processo.

3.4 Minimização de Dependências Externas

Um dos princípios orientadores do projeto foi a redução do uso de bibliotecas externas, com o objetivo de manter maior controle sobre o funcionamento interno da *engine* e aprofundar o entendimento sobre seus mecanismos estruturais.

Foram utilizadas apenas dependências estritamente necessárias para acesso a recursos de baixo nível, evitando *frameworks* de alto nível que encapsulam a lógica de renderização ou gerenciamento de entidades. Essa decisão está alinhada ao objetivo de explorar de forma direta os fundamentos técnicos envolvidos no desenvolvimento de motores gráficos.

3.5 Validação do Sistema

A validação da biblioteca foi realizada por meio do desenvolvimento de um protótipo funcional de jogo 2D baseado em *tiles*, utilizando visão *top-down*. O protótipo teve como finalidade verificar:

- Correta renderização dos mapas;
- Funcionamento do sistema de movimentação;
- Detecção básica de colisões;
- Gerenciamento adequado de recursos gráficos.

A análise dos resultados foi conduzida de forma qualitativa, observando desempenho, estabilidade e organização estrutural do código.

3.6 Limitações Metodológicas

Como o foco do trabalho está na arquitetura e implementação estrutural da *engine*, não foram realizados testes comparativos quantitativos com motores consolidados de mercado. A avaliação concentrou-se na funcionalidade, organização arquitetural e adequação ao escopo proposto.

REFERÊNCIAS

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

GIL, Antonio Carlos. Métodos e técnicas de pesquisa social. 7. ed. São Paulo: Atlas, 2019.

GREGORY, Jason. Game engine architecture. 3. ed. Boca Raton: CRC Press, 2018.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2020.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

STROUSTRUP, Bjarne. The C++ programming language. 4. ed. Boston: Addison-Wesley, 2013.